

Motivation 1 — Sparse-constrained GLM

The problem we want to solve

Table of contents

A familiar starting point: bodyfat	1
Generalising to GLMs	1
The optimisation we are trying to solve	2
Why it is hard	2
Where this matters	3
The plan from here	3

A familiar starting point: bodyfat

Suppose we want to predict body-fat percentage from physical measurements. The classical dataset (Penrose et al. 1985) records 14 measurements on 252 men:

Table 1: Bodyfat data — first 5 rows. Response **Bodyfat** (%) in last column.

Age	Weight	Height	Neck	Chest	Ab- domen	Hip	Thigh	Knee	An- kle	Bi- ceps	Fore- arm	Wrist	Body- fat
23	70.1	172.1	36.2	93.1	85.2	94.5	59.0	37.3	21.9	32.0	27.4	17.1	12.3
22	78.8	183.5	38.5	93.6	83.0	98.7	58.7	37.3	23.4	30.5	28.9	18.2	6.1
22	70.0	168.3	34.0	95.8	87.9	99.2	59.6	38.9	24.0	28.8	25.2	16.6	25.3
26	83.5	183.5	37.4	101.8	86.4	101.2	60.1	37.3	22.8	32.4	29.4	18.2	10.4
24	80.4	180.3	39.0	97.3	100.0	101.9	63.2	42.2	24.0	32.2	27.7	17.7	28.7

Which **handful** of these measurements is enough to predict body fat well? That is best subset selection.

Generalising to GLMs

The same question arises whenever the response is well modelled by a generalised linear model (GLM):

Response type	Family	Examples
Continuous	Gaussian	bodyfat, gene-expression intensity
Binary	Bernoulli (logistic)	disease yes/no, GWAS case/control
Multinomial	softmax (multinomial logistic)	cancer subtypes

Response type	Family	Examples
Count	Poisson	event counts

In every case, fitting the model amounts to maximising a log-likelihood:

$$\ell(\beta_0, \beta) = \sum_{i=1}^n \log p(y_i \mid x_i; \beta_0, \beta).$$

For Gaussian, this reduces to least squares; for Bernoulli, to logistic regression; and so on. The unifying framework lets one method handle them all — provided it can deal with the **sparsity constraint** we are about to add.

The optimisation we are trying to solve

For a prescribed model size k , find the k predictors that best fit the data:

$$\min_{\beta_0 \in \mathbb{R}, \beta \in \mathbb{R}^p} \left\{ -\frac{1}{n} \ell(\beta_0, \beta) + \lambda \|\beta\|_2^2 \right\} \quad \text{subject to} \quad \|\beta\|_0 \leq k.$$

Three things to notice:

- The objective is the **negative GLM log-likelihood**, optionally stabilised with a small ridge term $\lambda \|\beta\|_2^2$.
- The constraint is the ℓ_0 -**norm cardinality**: count the non-zero entries, keep at most k .
- The user supplies k . Output: which k features, and their fitted coefficients.

This is the problem COMBSS solves.

Why it is hard

Counting feasible subsets:

Setting	Candidate subsets
Bodyfat (p=13, k=5)	1,287
Khan SRBCT (p=2308, k=12)	4.64e+31
Rice GWAS (p=158210, k=10)	2.71e+45

For $p = 13$ enumeration is fine. For $p = 2308$ it is 10^{33} **subsets**; for $p \approx 1.6 \times 10^5$ it is 10^{45} . The combinatorial blowup makes exhaustive search hopeless within the lifetime of the universe.

In fact the problem is **NP-hard** (Natarajan 1995). No polynomial-time algorithm is expected for the worst case.

Where this matters

The setting $p \gg n$ with a sparse truth is the bread and butter of modern applied statistics:

- **Genomics** — GWAS with 10^5 SNPs, gene-expression panels with 10^3 – 10^4 probes.
- **Survey data** — patient registries with hundreds of recorded variables and a few hundred outcomes.
- **Sensor data** — many channels, few labelled trials.
- **Risk scoring** — interpretable models in medicine and insurance, where a 10-feature panel is deployable and a 500-feature panel is not.

Every one of these calls for a sparsity-constrained GLM, not just a regularised one.

The plan from here

How do existing methods cope with the NP-hard constraint? Two main strategies:

- **Lasso** ([next page](#)) — relax $\|\beta\|_0$ to the convex $\|\beta\|_1$. Fast and popular, but biased and indexed by λ , not k .
- **MIO** ([page after](#)) — solve the discrete problem directly with mixed-integer optimisation. Exact, but does not scale.

COMBSS sits between the two: continuous like the lasso, but on the *support indicator* rather than on β itself — and explicitly k -indexed.

[Next → Lasso](#)